



Proyecto Final: ConchoAdmin



Asignatura: Programación 1

Integrantes:

Darvin Méndez - Matricula: [2025-2024] - Rol: Líder del Equipo

Zoila García – Matricula: [2025-1514] - Rol: Diseño

Luis Alberto Moscoso - Matrícula [2025-2065] - Rol: SQA

Kevin Daniel Martinez Reyes - [Matricula: 2025- 1159] - Rol: DBA

Repositorio: <https://github.com/Kevin-D-Martinez/UNIDEVS/tree/main/ConchoAdmin>

Especificaciones del Sistema:

Lenguaje: Java 25/ Java Swing

Base de Datos: MySQL

IDE Recomendado: NetBeans IDE

Fecha: Abril 2026

Version: 1.0

Instrucciones para su uso:

1. Configurar la Base de Datos (MySQL).
2. Agregar las librerías de la carpeta "Librerías" al class path del proyecto.
3. Ejecutar el programa desde el archivo app.ConchoAdmin.java.
4. Iniciar sesión si tienes cuenta existente o registrarte haciendo clic en "Registrarte".

Presionar el logo de UNIDEVS debajo del botón Cerrar sesión para saber acerca de la aplicación y su uso.

1. Introduccion:

Cual es la problematica?

Actualmente, la gestión del transporte público se encuentra estancada en métodos analógicos. Los dueños de rutas dependen exclusivamente de registros manuales en papel, un sistema ineficiente que no solo facilita la pérdida de información crítica, sino que impide la toma de decisiones basada en datos reales, limitando el crecimiento y la seguridad del servicio

Justificación de la solución:

ConchoAdmin es un sistema de gestión de rutas de transporte público que permite administrar choferes, vehículos y rutas. Con el fin de que los dueños de rutas tengan un mayor manejo de sus ingresos y choferes.

2. Requisitos funcionales del sistema:

2.1 Requisitos de software:

Java versión 8 o superior.

MySQL server versión 7.7 o superior.

NetBeans, IDE recomendado para el desarrollo

2.2 requisitos de hardware:

Procesador: 1Ghz o superior.

Memoria RAM: 3gb mínimo.

Espacio en disco: mínimo 500mb disponibles.

Sistema operativo: Windows, Linux, macOS.

3. Configuración de la base de datos:

<u>Tabla</u>	<u>Descripción</u>	<u>Relaciones (FK)</u>
Usuarios	Almacena las credenciales (email, contraseña) y datos de perfil del personal administrativo.	N/A
Choferes	Contiene información personal (cedula, telefono) y laboral del conductor.	id_ruta, id_usuario
Vehiculos	Registra las unidades de transporte (matricula, marca, modelo, año).	id_chofer, id_ruta, id_usuario
Rutas	Define los trayectos disponibles y la tarifa base asignada a cada uno.	id_usuario
Pagos	Auditoría financiera que registra el monto, metodoPago y el estado de la transacción.	id_chofer, id_ruta, id_usuario

4. creacion de la base de datos:

Tabla Choferes:

```

1 CREATE TABLE Choferes(
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     nombre VARCHAR(25),
4     apellido VARCHAR(25),
5     cedula CHAR(11) UNIQUE,
6     telefono CHAR(10) UNIQUE,
7     estado VARCHAR(10),
8     id_ruta INT,
9     id_usuario INT,
10    fechaCreacion DATETIME DEFAULT CURRENT_TIMESTAMP,
11    FOREIGN KEY (id_ruta) REFERENCES Ruta(id),
12    FOREIGN KEY (id_usuario) REFERENCES Usuario(id)
13 )
14
15 SELECT * FROM Chofer
16
17 SELECT * FROM Chofer WHERE (nombre LIKE '%Ra%' OR apellido LIKE '%Ra%' OR cedula LIKE '%Ra%') AND id_usuario = 8

```

Tabla Pagos:

```

1 CREATE TABLE Pagos (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     monto DECIMAL(6,2),
4     metodoPago VARCHAR(10),
5     estado VARCHAR(10),
6     id_chofer INT,
7     id_ruta INT,
8     id_usuario INT,
9     fechaCreacion DATETIME DEFAULT CURRENT_TIMESTAMP,
10    FOREIGN KEY (id_chofer) REFERENCES Chofer(id),
11    FOREIGN KEY (id_ruta) REFERENCES Ruta(id),
12    FOREIGN KEY (id_usuario) REFERENCES Usuario(id)
13 )
14
15 SELECT * from Pago;

```

Tabla Rutas:

```

1 CREATE TABLE Rutas (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     nombre VARCHAR(25),
4     tarifa DECIMAL(6,2),
5     id_usuario INT,
6     fechaCreacion DATETIME DEFAULT CURRENT_TIMESTAMP,
7     FOREIGN KEY (id_usuario) REFERENCES Usuario(id)
8 )
9
10 SELECT * FROM Ruta;
11
12 KILL 96116;
13 SHOW PROCESSLIST;

```

Tabla Usuarios:

```

1 CREATE TABLE Usuarios (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     nombre VARCHAR(25),
4     apellido VARCHAR(25),
5     email VARCHAR(50) UNIQUE,
6     contraseña VARCHAR(15),
7     fechaCreacion DATETIME DEFAULT CURRENT_TIMESTAMP
8 )
9
10 SELECT * FROM Usuario;

```

Tabla Vehiculos:

```
1 CREATE TABLE Vehiculos (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     marca VARCHAR(15),
4     modelo VARCHAR(10),
5     año VARCHAR(4),
6     matricula CHAR(7),
7     id_chofer INT,
8     id_ruta INT,
9     id_usuario INT,
10    fechaCreacion DATETIME DEFAULT CURRENT_TIMESTAMP,
11    FOREIGN KEY (id_chofer) REFERENCES Chofer(id),
12    FOREIGN KEY (id_ruta) REFERENCES Ruta(id),
13    FOREIGN KEY (id_usuario) REFERENCES Usuario(id)
14 )
15
16 SELECT * FROM Vehiculo
```

5. Credenciales de acceso:

Host (Servidor): bua4hqkt6dimvqko4gzv-mysql.services.clever-cloud.com

Puerto: 3306

Base de Datos: bua4hqkt6dimvqko4gzv

Usuario: u40gkd53pb7ugtgz

Contraseña: db.contra=***** (Protegida por motivos de seguridad).

6. Estructura del proyecto:

6.1 Capa de Datos y Persistencia (modelo)

Esta capa es el núcleo del manejo de información y se subdivide para evitar la mezcla de lógica SQL con objetos de negocio.

modelo.DAO (Data Access Object): Contiene las clases encargadas de ejecutar las sentencias SQL. Cada archivo (ej. ChoferDAO.java, RutaDAO.java) actúa como un gestor que traduce las peticiones de Java a comandos que la base de datos.

modelo.DTO (Data Transfer Object): Define la estructura de los objetos que viajan por el sistema. Si la base de datos tiene una tabla "Vehiculos", en este paquete existe una clase Vehiculo.java con los mismos atributos para transportar esa información.

modelo.estructuraBD: Es el repositorio de configuración. Centraliza la clase ConexionMySQL.java y guarda los respaldos .sql de las tablas, permitiendo recrear la base de datos en cualquier momento.

6.2 Gestión de Sesión y Estado

SesionActiva.java: Ubicada en la raíz del paquete modelo, esta clase es vital para la seguridad y personalización. Permite que el sistema "recuerde" qué usuario ha iniciado sesión, facilitando que su id_usuario se guarde automáticamente en cada registro de choferes, rutas o pagos que realice.

6.3 Recursos y Multimedia (assets)

Este paquete centraliza todos los elementos visuales externos. Almacena:

Imágenes: Logos del sistema "ConchoAdmin".

Iconos: Gráficos para los botones de "Guardar", "Eliminar" o "Buscar".

6.4. Interfaz y Navegación (vistas)

Representa la capa superior del modelo MVC. Aquí se alojan los formularios (JFrames) y paneles (JPanels) que componen el dashboard principal y las ventanas de gestión.

6.5. Librerías Externas (Libraries)

Para que la estructura sea funcional, el proyecto incluye el MySQL Connector/J. Esta librería es el puente físico que utiliza ConexionMySQL.java para entablar comunicación con el puerto 3306 del servidor remoto.

7. Patron de Diseño:

Utilizamos el patrón **Modelo-Vista-Controlador (MVC)** ya que es la forma más profesional de pasar de un sistema de "lápiz y papel" a una aplicación escalable. Esto permitirá

separar la lógica de los datos de la interfaz que verán los dueños de las rutas.